Lctr 24: Combination Branching and Conditional Execution

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Combination branching.
- Conditional execution instructions.
- The IT (If-then) instruction for organizing the conditional execution instructions.
- Implementing C's `if-else` construct with conditional execution instructions.

Demo project:

- Applications of the above instructions.
- Contrast the implementation of C's `if-else` construct with conditional branch and conditional execution instructions.

24.1 Introduction

In Lctr 19, we have discussed the basic topics of condition code suffixes (CCS) and how they are used in forming conditional branch instructions, which are used to control the flow of the program.

In this lecture, we will extend our discussion to two classes of new instructions:

- The first class is the combination branch instructions, which combine a special compare operation and special conditional branch instructions into **one** instruction. This way, we can save the code size. Yet, we need to know the constraints when the combination branch instructions are used.
- The second class is the conditional execution instructions that are formed by attaching the CCS to many common instructions. This way, we will be able to execute these functions according to the status flags. Note that these instructions need to be organized under the IT instruction to put them in use.

24.2 Combination branch instructions

Some combinations of comparison and branch are so often used in real applications that ARM engineers have created combination branch instructions to simplify the programming of some of the operations. For example, we can use the following two combination branch instructions.

Instruction	Description
`CBZ Rn, label`	Compare and branch to `label` if `Rn` is zero
`CBNZ Rn, label`	Compare and branch to `label` if `Rn` is not zero

Note that:

- `CBZ Rn, label` is equivalent to

```
    CMP  Rn, #0
    BEQ  label
```

- `CBNZ Rn, label` is equivalent to

```
    CMP  Rn, #0
    BNE  label
```

- There are some limitations to the `CBZ` and `CBNZ` instructions, including:
  - Rn must be in the range of R0 and R7. These are the low registers, so we only need to have three bits to encode them—selecting one of them. This is needed due to packing the label info in a 32-bit instruction.
  - The branch destination, expressed by `label`, must be within 4 to 130 *after* the current instruction, which means we can only use a *positive offset*, which must be able to be expressed by 8 bits. Again, this is needed due to packing the label into a 32-bit instruction.

Example 1. Consider the following C function used to find the length of an array, **which includes the trailing string-ending null character, the true 0**:

```
int mp_str_len_c(char *str) {
    int len = 1;
    while (*str++) {
        len++;
    }
    return len;
}
```

Translate this C function into ASM functions using (a) the standard conditional branch instruction, (b) `CBZ`, and c) `CBNZ`, respectively. Note that we need to use the following prototypes, respectively:

```
int mp_str_len_cmp_s(char *str);
int mp_str_len_cbz_s(char *str);
int mp_str_len_cbnz_s(char *str);
```

Solution:

Part (a).

```
@ int mp_str_len_cmp_s(char *str);
@   Input:      *str -> r0
@   Output:     r0
mp_str_len_cmp_s:
    ldr     r1, =1          @ len = 1
str_len_cmp_s_begin:
    ldrb    r2, [r0], #1    @ *str++
    cmp     r2, #0
    beq     str_len_cmp_s_end
    add     r1, #1
    b       str_len_cmp_s_begin
str_len_cmp_s_end:
    mov     r0, r1
    bx      lr
```

Part (b).

```
@ int mp_str_len_cbz_s(char *str);
mp_str_len_cbz_s:
    ldr     r1, =1          @ len = 1
```

```
str_len_cbz_s_begin:
    ldrb    r2, [r0], #1  @ *str++
    cbz     r2, str_len_cbz_s_end
    add     r1, #1
    b       str_len_cbz_s_begin
str_len_cbz_s_end:
    mov     r0, r1
    bx      lr
```

Part (c).

```
@ int mp_str_len_cbnz_s(char *str);
mp_str_len_cbnz_s:
    ldr     r1, =1        @ len = 1
str_len_cbnz_s_begin:
    ldrb    r2, [r0], #1  @ *str++
    cbnz    r2, str_len_cbnz_s_update
    mov     r0, r1
    bx      lr
str_len_cbnz_s_update:
    add     r1, #1
    b       str_len_cbnz_s_begin
```

> -:-

24.3 Conditional execution instructions

24.3.1 Syntax of conditional execution instructions

Below, we use the `ADD` instruction to illustrate the syntax of conditional execution instructions, which can be written as:

```
    ADD{S}{cond} Rd, Rn, Op2
```

where

- `{S}` is the optional suffix for updating the status flags.
- `{cond}` is the optional CCS.

Here are the commonly used conditional *addition* instructions, which do not update the status flags, based on the CCSs we discussed before:

Relational test	Signed instructions	Unsigned instructions
==	ADDEQ	ADDEQ
!=	ADDNE	ADDNE
>	ADDGT	ADDHI
>=	ADDGE	ADDHS
<=	ADDLE	ADDLS
<	ADDLT	ADDLO

For conditional execution, we have the following notes:

- The CCS enables the processor to test a condition, which is often a relation between two numbers, based on the condition (status) flags.
- If the condition test of a conditional instruction fails, the instruction **does not**
  - execute,
  - write any value to its destination register,
  - affect any condition flags, and
  - generate an exception.

While the conditional execution instructions can be used directly in the ARM toolchain, they have to be used with the help of the IT (If-Then) instruction, which is discussed below, for the GNU toolchain. In fact, the ARM toolchain will add the correct form of the IT instruction automatically.

24.3.2 The IT (If-Then) instruction

It is easier to give an example of the IT instructions before discussing its full and flexible format.

Example 2. Reimplement the above `str_len_cmp_s` function using conditional execution with the following prototype:

```
int mp_str_len_cond_s(char *str)
```

Solution:

```
@ int mp_str_len_cond_s(char *str);
mp_str_len_cond_s:
    ldr     r1, =1        @ len = 1
str_len_cond_s_begin:
    ldrb    r2, [r0], #1  @ *str++
    cmp     r2, #0
    itt     ne
    addne   r1, #1
    bne     str_len_cond_s_begin
str_len_cond_s_end:
    mov   r0, r1
    bx    lr
```

> -:-

Now, let's take a look at the generic form of the IT instruction, which can cover up to 4 conditional execution instructions using the following syntax:

```
IT{x{y{z}}} cond
```

- `I` stands for **if**.
- `T` stands for **then**. This is for the required (first) conditional instruction.
- `cond` is one of the CCSs we have discussed so far.
- `x`, `y`, and `z` specify the existence of the optional second, third, and fourth conditional instructions, respectively. They can be either `T`, for **then**, or `E`, for **else**.
- The conditional execution instruction corresponding the `T` must have the CCS that is the same as `cond`.
- The conditional execution instruction corresponding the `E` must have the CCS that is the inverse of `cond`.

About the syntax of the IT instruction, we have the following notes:

- The **conditional branch** instruction can run without the IT instruction, as we have been using. It can also be at the end of the IT block, like the one we have just seen. **It cannot be in the middle of the IT block** though, as this simply does not make sense.
- The CBZ and CBNZ instructions cannot be used as conditional execution instructions within the IT block.
- There are additional restrictions to the IT block, which will not be listed here. See p 145 of [PM0214] for details if you are curious or have compilation issues.

24.4 Examples of conditional branch and execution

The following examples use the memory map shown in Figure 1. Also, we assume

```
int x = 1, y = -1;
int *p = (int *)0x20000204;
```

	Byte 0	Byte 1
0x2000_0200	0x70	0x72
0x2000_0202	0x74	0x76
0x2000_0204	0x78	0x7A
0x2000_0206	0x7C	0x7E
0x2000_0208	0x80	0x82
0x2000_020A	0x84	0x86
0x2000_020C	0x88	0x8A
0x2000_020E	0x8C	0x8E

Figure 1: Memory map.

Example 3. (a) What will be the return value if we run the program below?

```
@ int mp_x_cmp_y_i_load_cmp(int x, int y, void *p);
@   Input:      x -> r0, y -> r, *p -> r2
@   Output:     r0
mp_x_cmp_y_i_load_cmp:
    cmp     r0, r1
    ble     x_cmp_y_i_load_cmp_else @ Negative logic!
x_cmp_y_i_load_cmp_then:
    ldrsb   r0, [r2, #4]
    b       x_cmp_y_i_load_cmp_end
x_cmp_y_i_load_cmp_else:
    ldrsh   r0, [r2], #4
x_cmp_y_i_load_cmp_end:
    bx      lr
```

(b) Rewrite the above function using condition execution with the following prototype

```
@ int mp_x_cmp_y_i_load_cond(int x, int y, int *p);
```

Solution:

b):

```
@ int mp_x_cmp_y_i_load_cond(int x, int y, void *p);
@   Input:      x -> r0, y -> r, *p -> r2
@   Output:     r0
mp_x_cmp_y_i_load_cond:
    cmp     r0, r1
    ite     gt
    ldrsbgt r0, [r2, #4]
    ldrshle r0, [r2], #4
    bx      lr
```

> -:-

Example 4. (a) What will be the return value if we run the program below?

```
@ int mp_x_cmp_y_u_load_cmp(uint32_t x, uint32_t y, int *p);
@   Input:      x -> r0, y -> r, *p -> r2
@   Output:     r0
mp_x_cmp_y_u_load_cmp:
    cmp     r0, r1
    bls     x_cmp_y_u_load_cmp_else
x_cmp_y_u_load_cmp_then:
    ldrh    r0, [r2, #4]!
    b       x_cmp_y_u_load_cmp_end
x_cmp_y_u_load_cmp_else:
    ldr     r0, [r2, #4]
x_cmp_y_u_load_cmp_end:
    bx      lr
```

(b) Rewrite the above function using condition execution with the following prototype

```
; int mp_x_cmp_y_u_load_cond(uint32_t x, uint32_t y, int *p);
```

Solution:

(b):

```
@ int mp_x_cmp_y_u_load_cond(uint32_t x, uint32_t y, int *p);
@   Input:      x -> r0, y -> r, *p -> r2
@   Output:     r0
mp_x_cmp_y_u_load_cond:
    cmp     r0, r1
    ite     hi
    ldrhhi  r0, [r2, #4]!
    ldrls   r0, [r2, #4]
    bx      lr
```

> -:-

24.5 Exercises

Solutions of the following exercises can be found in the demo project.

Exercise 1. Convert the following ASM code from conditional branch to conditional execution based on the `MOV` instruction.

```
@ int mp_x_cmp_y_i_ab_cmp(int x, int y, int a, int b);
@   Input:      x -> r0, y -> r1, a -> r2, b -> r3
@   Output:     r0
mp_x_cmp_y_i_ab_cmp:
    cmp     r0, r1
    ble     mp_x_cmp_y_i_ab_cmp_else
mp_x_cmp_y_i_ab_cmp_then:
    mov     r0, r2
    b       mp_x_cmp_y_i_ab_cmp_end
mp_x_cmp_y_i_ab_cmp_else:
    mov     r0, r3
mp_x_cmp_y_i_ab_cmp_end:
    bx      lr
```

Exercise 2. Convert the following ASM code from conditional branch to conditional execution based on the `MOV` instruction.

```
@ int mp_x_cmp_y_u_ab_cmp(uint32_t x, uint32_t y, int a, int b
    );
@   Input:      x -> r0, y -> r1, a -> r2, b -> r3
@   Output:     r0
mp_x_cmp_y_u_ab_cmp:
    cmp     r0, r1
    bls     mp_x_cmp_y_u_ab_cmp_else
mp_x_cmp_y_u_ab_cmp_then:
    mov     r0, r2
    b       mp_x_cmp_y_u_ab_cmp_end
mp_x_cmp_y_u_ab_cmp_else:
    mov     r0, r3
mp_x_cmp_y_u_ab_cmp_end:
    bx      lr
```

Exercise 3. Write two ASM functions for calculating the absolute value of an integer using the following prototypes using the conditional branch and the conditional execution, respectively.

```
@ int mp_abs_cmp(int x);
@ int mp_abs_cond(int x);
```